



FIT Lecture # 6

Engr. Naveera Sami



PROGRAMMING LANGUAGES:

Computers can perform a variety of tasks. However, they cannot perform any of them on their own. As we know, computers have no commonsense and they cannot think. They need clear-cut instructions to tell them what to do, how to do and when to do. A set of instructions to carry out these functions is called a computer program.

The communication between two parties, whether they are machines or human beings, always needs a common language or terminology. The language used in the communication of instructions to a computer is known as computer language or programming language. There are many different types of languages available today. A computer program can be written using any of the programming languages depending upon the task to be performed and the knowledge of the person developing the program. The process of writing instructions using a computer language is known as programming or coding. The person who writes such instructions is referred as a programmer.

HISTORY OF PROGRAMMING LANGUAGES:

The history of programming languages is interlinked with the evolution of computer systems. As the computer systems became smaller, faster and cheaper with time, the programming languages also became more and more user friendly.

Ada Augusta Lovelace, a companion of Charles Babbage, was considered as the first computer programmer in the history of programming languages. In the year 1843, Ada Augusta Lovelace wrote a set of instructions to program the analytical engine designed by Charles Babbage.

This computer program was used to transform the data entered by the users into binary form before being processed by the computer system. This program increased the efficiency and the productivity of the analytical engine by automating various mathematical tasks. Later, in the year 1946, Konrad Zuse, a German engineer, developed a programming language known as Plankalkul. It was developed to target the various scientific, business and engineering needs of the users. It was considered as the first complete programming language that supported various programming constructs and the concept of data structures as well. The various programming constructs were implemented in this programming language with the help of Boolean algebra.

During the 1940s, machine languages were developed to program the computer system. The machine languages which used binary codes 0s and 1s to represent instructions were regarded as low-level programming languages. The instructions written in the machine language could be executed directly by the CPU of the computer system. These languages were hardware dependent languages. Therefore, it was not possible to run a program developed for one computer system in another computer system. This is because of the fact that the internal architecture of one computer system may be different from that of another. The development of programs in machine languages was not an easy task for the programmers. One was required to have thorough knowledge of the internal architecture of the computer system before developing a program in machine language.

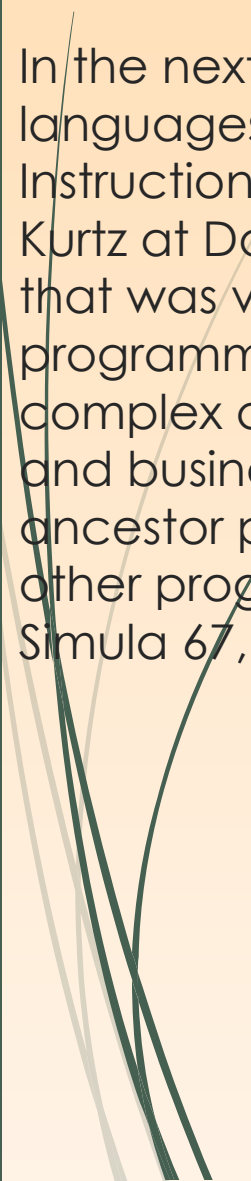
During the 1950s, assembly language, which is another low-level programming language, was developed to program the computer systems. The assembly language used the concept of mnemonics to write the instructions of a computer program. Mnemonics refer to the symbolic names that are used to replace the machine language code. The programmers enjoyed working with this programming language because it was easy to develop a program in the assembly language as compared to the machine language. However, unlike machine language programs, assembly language programs could not be directly executed by the CPU of the computer system and required some software program to convert these programs into machine understandable form.

During the period between 1950 and 1960, many high-level programming languages were developed to cater to the needs of the users of various disciplines, such as business, science and engineering. In 1951, Grace Hopper, an American computer scientist, started working towards designing a compiler called A-0, and in the year 1957, developed a high-level programming language known as MATH-MATIC. In 1952, another programming system known as AUTOCODE was developed by Alick E. Glennie. Grace Hopper was considered as the first person who had put some serious efforts towards the development of a high-level programming language.

In the year 1957, another popular high-level programming language known as FORTRAN (FORMula TRANslation) was developed. During its era, it was the only high-level programming language that became hugely popular among its users. FORTRAN was developed by John Backus and his team at International Business Machines (IBM). FORTRAN was best suited for solving problems related to scientific and numerical analysis field. Another high-level programming language known as ALGOL (Algorithm Language) was developed in the year 1958. Some other high-level languages that evolved during this era were LIST Processing (LISP) in 1958, Common Business Oriented Language (COBOL) in 1959 and ALGOL 60 in 1960.



In the next decade, from 1960 to 1970, more high-level programming languages evolved. In the year 1964, the Beginners All-Purpose Symbolic Instruction Code (BASIC) was designed by John G. Kemeny and Thomas E. Kurtz at Dartmouth college. It was a general-purpose programming language that was very simple to use. In the same year, another powerful high-level programming language, PL/I with many rich programming features such as complex data type and methods, was designed for developing engineering and business applications. PL/I was considered to have the best features of its ancestor programming languages: COBOL, FORTRAN, and ALGOL 60. The other programming languages that evolved during this era were Simula I, Simula 67, Algol 68 and APL.



The period between 1970 and 1980 was actually the golden era for the development high-level programming languages. This period saw the birth of many general-purpose and powerful high-level programming languages. In the early 1970s, a procedural programming language, Pascal was developed by Niklaus Wirth. This programming language was provided with strong data structures and pointers, which helped in utilizing the memory of the computer system in an efficient manner. In the year 1972, Dennis Ritchie developed a powerful procedural and block structured programming language known as C. C is still very popular among the users for developing system as well as application software. In 1974, IBM developed Structured Query Language (SQL) that was used for performing various operations on the databases, such as creating, retrieving, deleting and updating. Apart from these programming languages, some other high-level programming languages that evolved during this era were Forth, Smalltalk, and Prolog.

During the next decade, from 1980 to 1990, the focus of development of high-level programming languages shifted towards enhancing the performance and design methodology. The languages of this period used modular approach for designing large-scale applications. The modular approach of program design can be regarded as a design methodology, which divides the whole system into smaller parts that could be developed independently. This approach of designing software applications is still employed by modern programming languages. Some of the high-level programming languages that evolved during this era include Ada, C++, Perl and Eiffel

The high-level programming languages developed and designed in the 1990s are considered as the fifth generation programming languages. During this period, Internet technology evolved tremendously. Therefore, the basic purpose of the programming languages of this period was to develop web-based applications. However, these languages could also be used for the development of desktop applications. The important high-level programming languages of this era are Java, VB and C#. Most of the programming languages of this era employed object-oriented programming paradigm for designing and developing robust and reliable software applications.

Period of employment	Programming language	Characteristics
1940s	Machine language	• Machine dependent • Faster execution • Difficult to use and understand • More prone to errors
1950s	Assembly language	• Machine dependent • Faster execution • More prone to errors • Relatively simple to use
1950–1970	FORTRAN, LISP, COBOL, ALGOL 60, BASIC, APL	• High-level languages • Easy to develop and understand programs • Less prone to errors
1970–1990	C, C++, Forth, Prolog, Smalltalk, Ada, Perl, SQL	• Very high-level languages • Easier to learn • Highly portable
1990s	Java, HTML, VB, PHP, XML, C #	• Internet-based languages • Object-oriented languages • More efficient • Reliable and robust

GENERATIONS OF PROGRAMMING LANGUAGES

Programming languages have been developed over the years in a phased manner. Each phase of development has made the programming languages more user-friendly, easier to use and more powerful.

Each phase of improvement made in the development of the programming languages can be referred as a generation. The programming languages, in terms of their performance, reliability and robustness can be grouped into five different generations.

1. First generation languages (1GL)
2. Second generation languages (2GL)
3. Third generation languages (3GL)
4. Fourth generation languages (4GL)
5. Fifth generation languages (5GL)

First Generation: Machine Languages

The first generation programming languages are also called low-level programming language because they were used to program the computer systems at a very low level of abstraction, i.e., at the machine-level. The machine language, also referred as the native language of the computer system, is the first generation programming language. In the machine language, a programmer can issue the instructions to the computer system in the binary form only. Therefore, machine language programming only deals with two numbers, 0 and 1. The machine language programs are entered into the computer system by setting the appropriate switches available in the front panel system. These switches are actually the devices used to alter the course of the flow of electric current. The enable state of the switch represents the binary value, 1 and the disable state of the switch represents the binary value, 0. The programs written in the machine language are directly executed by the CPU of the computer system and therefore, unlike the modern programming languages, there is no need of using a translator in a machine language. Figure shows the typical instruction format of the machine language instruction.



Fig. 13.1 Machine instruction format

First Generation: Machine Languages

- The advantages of the first generation programming languages are:
- They are translation free and can be directly executed by the computers.
- The programs written in these languages are executed very speedily and efficiently by the CPU of the computer system.
- The programs written in these languages utilise the memory in an efficient manner because it is possible to keep track of each bit of data.

First Generation: Machine Languages

There are many disadvantages of using the first generation programming languages. They include:

- It is very difficult to develop a program in the machine language.
- The programs developed in these languages cannot be understood very easily by a person, who has not actually developed these programs.
- The programs written in these languages are so prone to frequent errors that they are very difficult to maintain.
- The errors in the programs developed in these languages cannot be detected and corrected easily.
- A programmer has to write a large number of instructions for executing even a simple task in these languages. Therefore, we can say that these languages result in poor productivity while developing programs.
- The programs developed in these languages are hardware dependent and thus they are non-portable. Due to these limitations, machine languages are very rarely used for developing application programs.

Second Generation: Assembly Languages

- Like the first generation programming languages, the second generation programming languages also belong to the category of low-level programming languages. The second generation programming languages comprise of assembly languages that use the concept of mnemonics for writing programs. Similar to the machine language, the programmer of assembly language needs to have internal knowledge of the CPU registers and the instructions set before developing a program. In the assembly language, symbolic names are used to represent the opcode and the operand part of the instruction. For example, to move the contents of the CPU register, a1 to another CPU register, b1 the following assembly language instruction can be used:
 - `mov b1, a1`
 - The above code shows the use of symbolic name, `mov` in an assembly language instruction. The symbolic name, `mov` instructs the processor to transfer the data from one register to another. Using this symbolic name, a value can also be moved to a particular CPU register

Second Generation: Assembly Languages

The use of symbolic names made these languages little bit user-friendly as compared to the first generation programming languages. However, the second generation languages were still machine dependent. Therefore, one was required to have adequate knowledge of the internal architecture of the computer system while developing programs in these languages.

Unlike the machine language programs, the programs written in the assembly language cannot be directly executed by the CPU of the computer system because they are not written in the binary form. As a result, some mechanism is needed to convert the assembly language programs into the machine understandable form. A software program called assembler is used to accomplish this purpose. An assembler is a translator program that converts the assembly language program into the machine language instructions. Figure 13.2 shows the role of an assembler in executing an assembly language program.



Fig. 13.2 Functioning of an assembler

Second Generation: Assembly Languages

□ The following are some of the advantages of second generation programming languages:

□ It is easy to develop, understand and modify the programs developed in these languages as compared to those developed in the first generation programming languages.

□ The programs written in these languages are less prone to errors, and therefore can be maintained with great ease.

□ The detection and correction of errors is relatively easy in these languages in comparison to the first generation programming languages.

Second Generation: Assembly Languages

The following are some of the disadvantages of the second generation programming languages:

- ❑ The programs developed in these languages are not executed as quickly as the programs developed in the machine language. This is because of the fact that the computer system needs to convert these programs into machine language before executing them.
- ❑ The programs developed in these languages are not portable as these languages are machine dependent.
- ❑ The programmer of these languages needs to have thorough knowledge of the internal architecture of the CPU for developing a program.
- ❑ The assembly language programs, like the machine language programs, still result in poor productivity.

Third Generation: High-level Languages

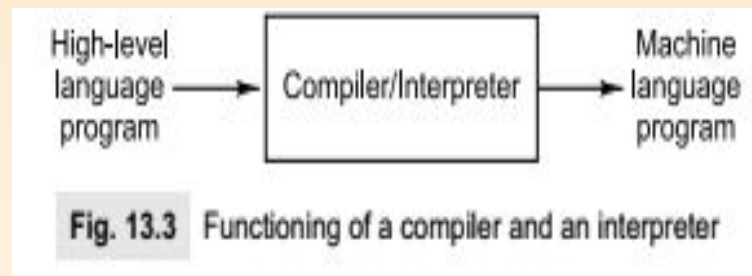
The third generation programming languages were designed to overcome the various limitations of the first and second generation programming languages. The languages of the third and later generations are considered as high-level programming languages because they enable the programmer to concentrate only on the logic of the program without concerning about the internal architecture of the computer system. In other words, we can also say that these languages are machine independent languages.

□ The third generation programming languages are also quite user-friendly because they relieve the programmer from the burden of remembering operation codes and instruction sets while writing a program. The instructions used in the third and later generations of languages can be specified in English like sentences, which are easy to comprehend for a programmer. The programming paradigm employed by most of the third generation programming languages was procedural programming, which is also known as imperative programming. In the procedural programming paradigm, a program is divided into a large number of procedures, also known as subroutines. Each procedure contains a set of instructions for performing a specific task. A procedure can be called by the other procedures while a program is being executed by the computer system.

Third Generation: High-level Languages

The third generation programming languages were considered as domain-specific programming languages because they were designed to develop software applications for a specific field. For example, the third generation programming language, COBOL, was designed to solve a large number of problems related to the business field only.

Unlike the assembly language, the programs developed in the third and the later generation of programming languages were not directly executed by the CPU of the computer system. These programs require translator programs for converting them into machine language. There are two types of translator programs, namely, compiler and interpreter. Figure 13.3 shows the translation of a program developed in the high-level programming language into the machine language program.



Third Generation: High-level Languages

- The following are some of the popular third generation programming languages:

FORTRAN, ALGOL ,BASIC, COBOL, C/C++

- The following are some of the advantages of the third generation programming languages:
- It is easy to develop, learn and understand the programs.
- The programs developed in these languages are highly portable as compared to the programs developed in the first and second generation programming languages. Hence, we can also say that the third generation programming languages are machine independent programming languages.
- The programs written in these languages can be developed in very less time as compared to the first and second generation programming languages. This is because of the fact that the third generation programming languages are quite user-friendly and provide necessary inbuilt tools required for developing an application.
- As the programs written in these languages are less prone to errors, they are easy to maintain.
- The third generation programming languages provide more enhanced documentation and debugging techniques as compared to the first and the second generation programming languages.

Third Generation: High-level Languages

The following are some of the disadvantages of the third generation programming languages:

As compared to the assembly and the machine language programs, the programs written in the third generation programming languages are executed slowly by the computer system.

The memory requirement of the programs written in these programming languages is more as compared to the programs developed using the assembly and machine languages.

Thanks

